



Instituto Tecnológico Superior de Lerdo

DESARROLLO DE APLICACIONES PARA DISPOSITIVOS MÓVILES

UNIDAD 3

Proyecto Unidad 3 – Tu primera aplicación de Flutter

Nombre del Alumno(s)

Jesús Antonio Arellano Zapata

Número de Control

202310117

Docente:

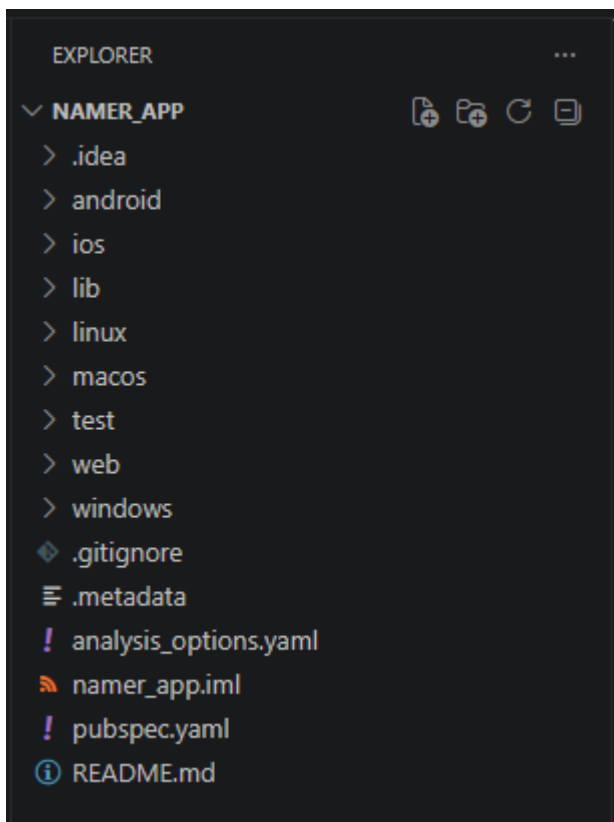
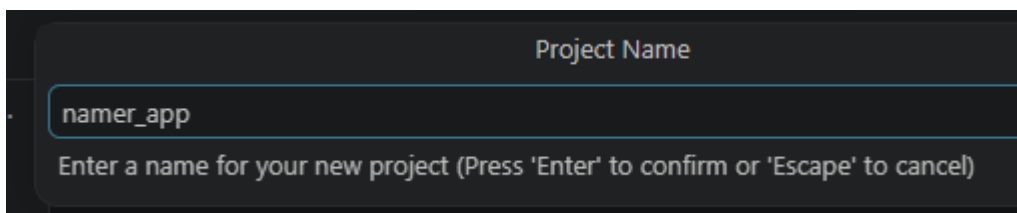
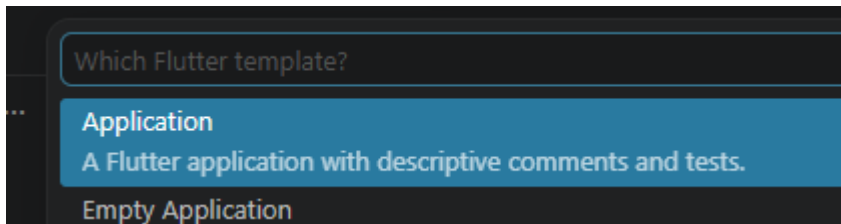
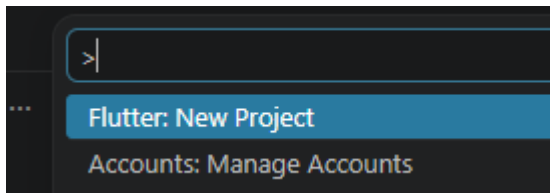
Jesús Salas Marín

Fecha de Entrega

12/06/2026

Creando el proyecto de Flutter en Visual Studio Code

En este paso creé un nuevo proyecto de Flutter desde Visual Studio Code, utilizando la opción **Flutter: New Project**. Después seleccioné el tipo de proyecto como aplicación y le asigné un nombre para generar la estructura inicial del proyecto.



Configurando la app inicial con los archivos proporcionados

```
! pubspec.yaml ●
! pubspec.yaml
1  name: namer_app
2  description: A new Flutter project.
3
4  publish_to: 'none' # Remove this line if
5
6  version: 0.0.1+1
7
8  environment:
9    sdk: '>=2.19.4 <4.0.0'
10
11  dependencies:
12    flutter:
13      sdk: flutter
14
15    english_words: ^4.0.0
16    provider: ^6.0.0
17
18  dev_dependencies:
19    flutter_test:
20      sdk: flutter
21
22    flutter_lints: ^2.0.0
23
24  flutter:
25    uses-material-design: true
26
```

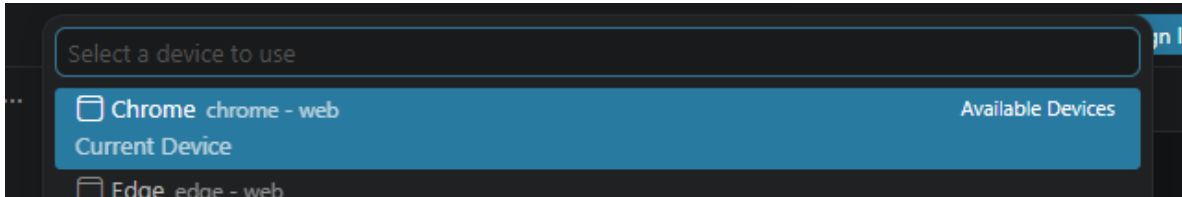
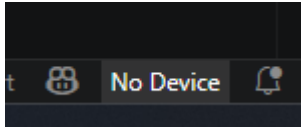
```
! pubspec.yaml  ! analysis_options.yaml X
! analysis_options.yaml
1  include: package:flutter_lints/flutter.yaml
2
3
4  linter:
5    rules:
6      prefer_const_constructors: false
7      prefer_final_fields: false
8      use_key_in_widget_constructors: false
9      prefer_const_literals_to_create_immutables: false
10     prefer_const_constructors_in_immutables: false
11     avoid_print: false
12
```

```
! pubspec.yaml  ! analysis_options.yaml  main.dart ●
lib > main.dart > ...
1  import 'package:english_words/english_words.dart';
2  import 'package:flutter/material.dart';
3  import 'package:provider/provider.dart';
4
5
6  Run | Debug | Profile
7  void main() {
8    runApp(MyApp());
9  }
10
11  class MyApp extends StatelessWidget {
12    const MyApp({super.key});
13
14
15    @override
16    Widget build(BuildContext context) {
17      return ChangeNotifierProvider(
18        create: (context) => MyAppState(),
19        child: MaterialApp(
20          title: 'Namer App',
21          theme: ThemeData(
22            useMaterial3: true,
23            colorScheme: ColorScheme.fromSeed(seedColor:
24            ), // ThemeData
25            home: MyHomePage(),
26          ), // MaterialApp
27        ); // ChangeNotifierProvider
28    }
29  }
```

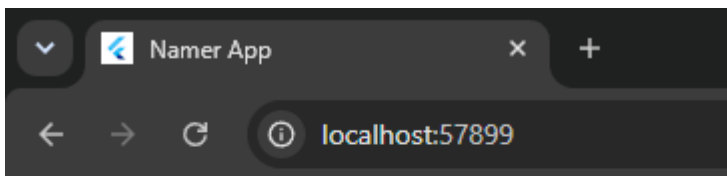
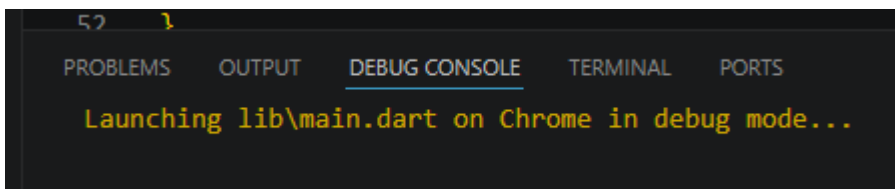
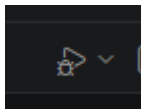
Ejecutando la aplicación por primera vez

En este paso ejecuté la aplicación en Visual Code. Esto permitió comprobar que el proyecto se iniciara correctamente en el dispositivo o navegador seleccionado.

Seleccionando el dispositivo de destino



Iniciando la aplicación



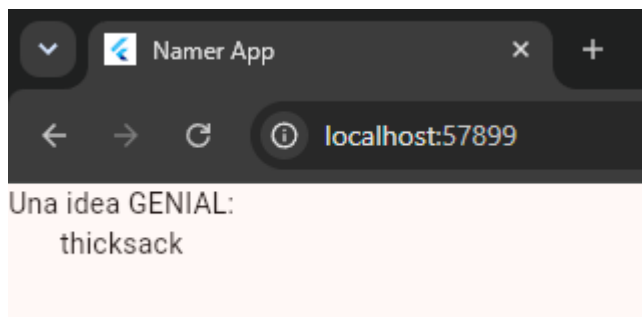
A random idea:
thicksack

Probando la recarga en caliente

En este paso realicé un pequeño cambio en el archivo main.dart y guardé el archivo para comprobar el funcionamiento de la recarga en caliente. Esto permitió ver los cambios reflejados en la aplicación sin tener que reiniciarla completamente.

```
42
43     return Scaffold(
44       body: Column(
45         children: [
46           Text('Una idea GENIAL:'),
47           Text(appState.current.asLowerCase),
48         ],
49       ), // Column
50     ); // Scaffold
51 }
```

```
Reloading application from main.dart
Reloaded application in 704ms.
```



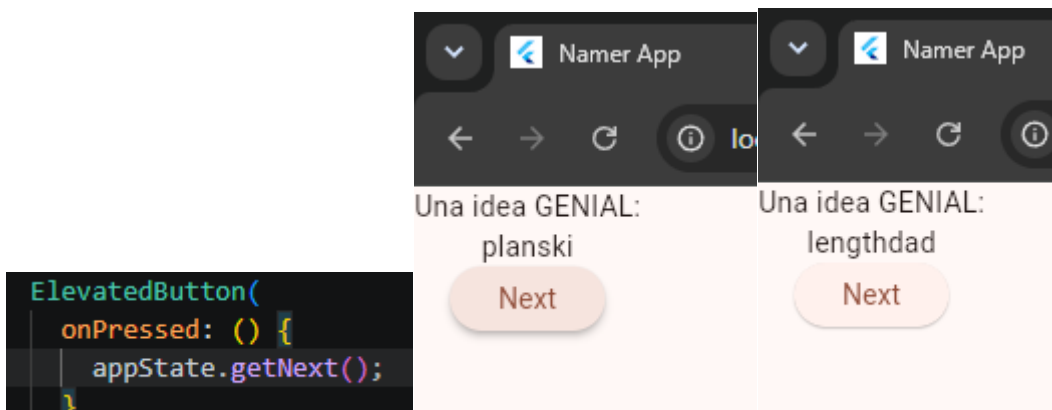
Agregando el botón Next

En este paso agregué un botón llamado Next

```
ElevatedButton(
  onPressed: () {
    print('Botón presionado :');
  },
  child: Text('Next'),
), // ElevatedButton
],
), // Column
```

A este botón se le dio la función generar un nuevo par de palabras aleatorias cada vez que el usuario lo presiona. Con esto la aplicación dejó de mostrar siempre la misma palabra y comenzó a tener interacción básica.

```
void getNext() {  
  current = WordPair.random();  
  notifyListeners();  
}
```



Creando el widget BigCard

En este paso separé el texto principal en un nuevo widget llamado BigCard. Esto permitió organizar mejor el código y hacer que la parte visual más importante de la aplicación estuviera separada del resto de la interfaz.

```
class MyHomePage extends StatelessWidget {  
  Widget build(BuildContext context) {  
    var appState = context.watch<MyAppState>();  
    var pair = appState.current;  
  
    return Scaffold(  
      body: Column(  
        children: [  
          Text('Una idea GENIAL:'),  
          Text(pair.asLowerCase),  
        ],  
      ),  
    );  
  }  
}  
  
class BigCard extends StatelessWidget {  
  const BigCard({  
    super.key,  
    required this.pair,  
  });  
  
  final WordPair pair;  
  
  @override  
  Widget build(BuildContext context) {  
    return Text(pair.asLowerCase);  
  }  
}
```

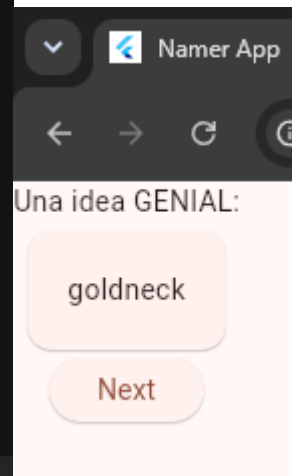
```
Text('Una idea GENIAL:'),  
BigCard(pair: pair),
```

Agregando espacio interno al texto, usando Padding para que el par de palabras no quede tan pegado a los bordes.

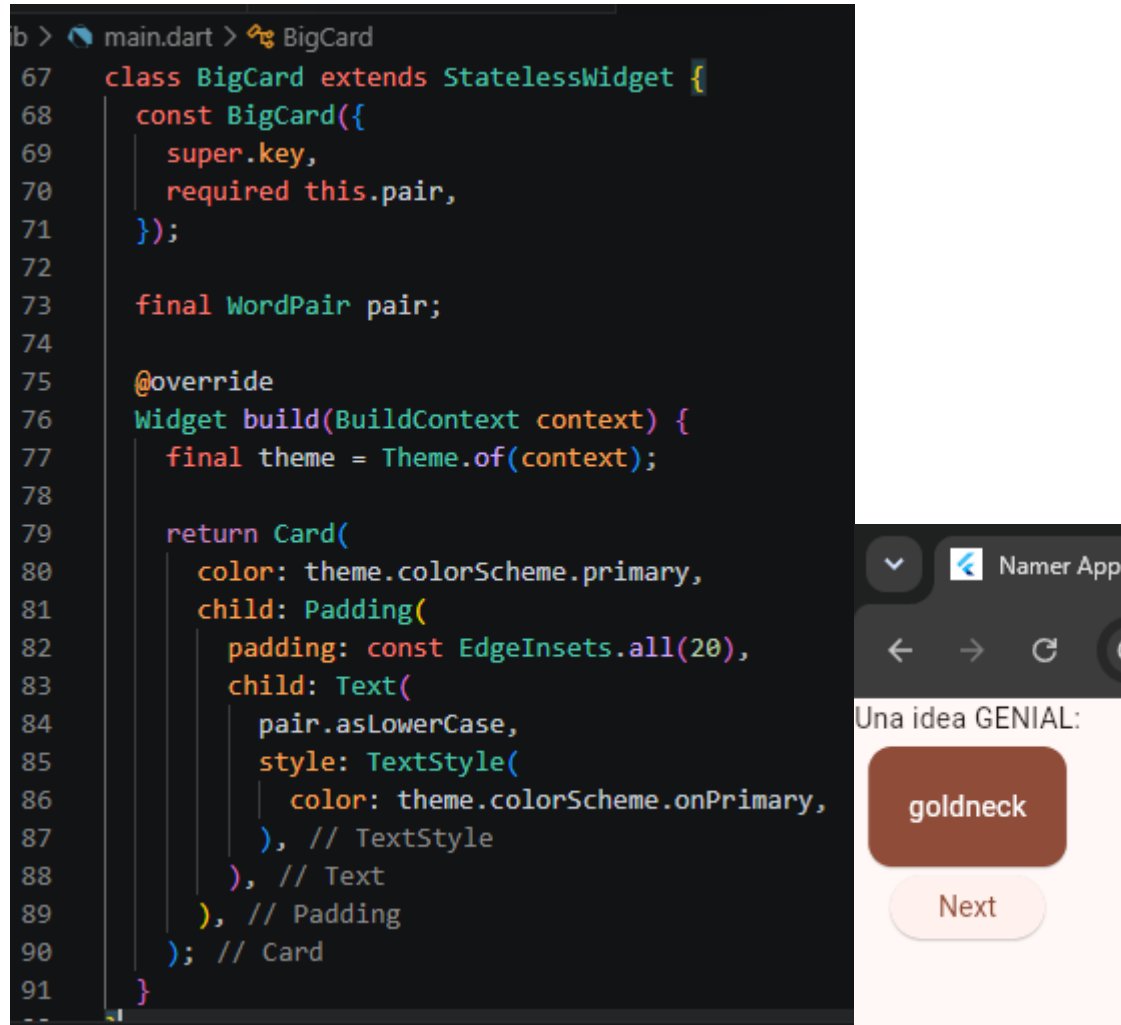
```
Widget build(BuildContext context) {  
  return Padding(  
    padding: const EdgeInsets.all(20),  
    child: Text(pair.asLowerCase),  
  ); // Padding  
}
```

Agregando una tarjeta para mostrar la palabra, colocando el texto dentro de un widget Card para que se vea más llamativo.

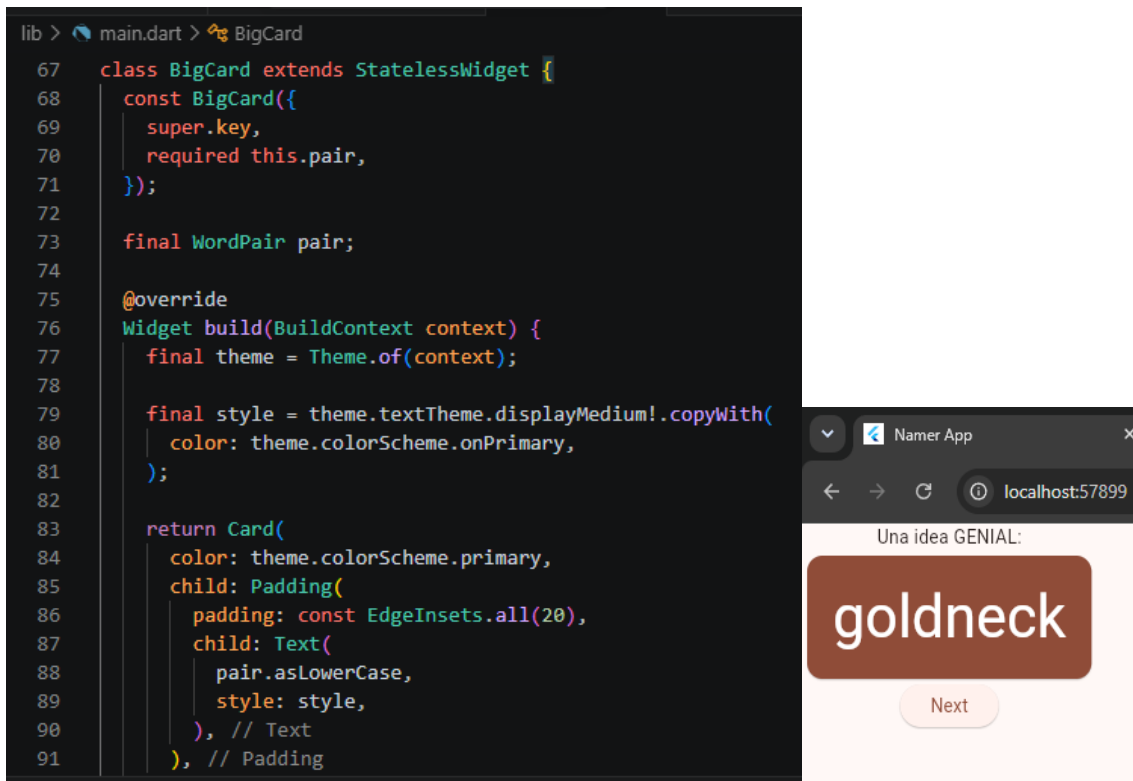
```
main.dart > BigCard  
67 class BigCard extends StatelessWidget {  
68   const BigCard({  
69     super.key,  
70     required this.pair,  
71   });  
72  
73   final WordPair pair;  
74  
75   @override  
76   Widget build(BuildContext context) {  
77     return Card(  
78       child: Padding(  
79         padding: const EdgeInsets.all(20),  
80         child: Text(pair.asLowerCase),  
81       ), // Padding  
82     ); // Card  
83   }  
84 }
```



Aplicando color a la tarjeta, usando los colores del tema de Flutter para que el diseño se vea más integrado.

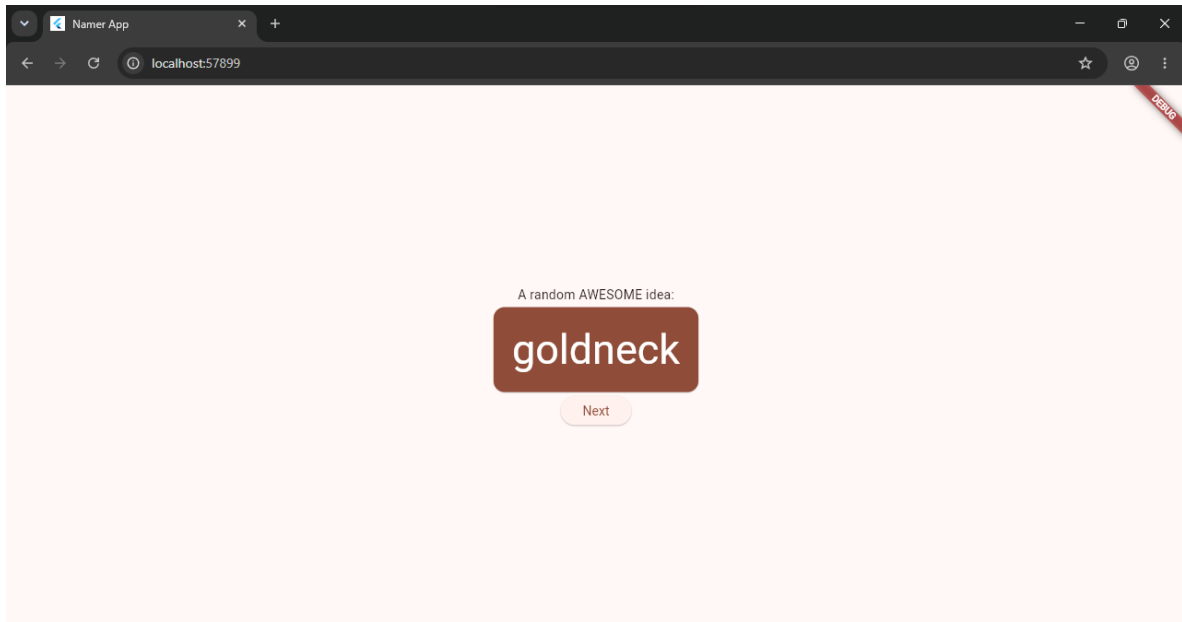


Aumentando el tamaño del texto principal, aplicando un estilo más grande para que el par de palabras sea más visible.



Centrando el contenido principal, usando Center y ajustando la columna para que los elementos aparezcan en medio de la pantalla.





Agregando espacio entre los elementos, usando `SizeBox` para separar la tarjeta del botón `Next`.

```
Text('A random AWESOME idea:'),  
BigCard(pair: pair),  
SizeBox(height: 10),  
ElevatedButton(  
  text: 'Next',  
  onPressed: () {  
    generateName();  
  },  
)
```



Agregando la funcionalidad

Agregando una lista de favoritos, creando una colección dentro de MyAppState para guardar los pares de palabras seleccionados.

```
class MyAppState extends ChangeNotifier {  
  var current = WordPair.random();  
  var favorites = <WordPair>[];  
  
  void getNext() {  
    current = WordPair.random();  
    notifyListeners();  
  }  
}
```

Creando el método para controlar favoritos, agregando una función que permita guardar o eliminar el par de palabras actual.

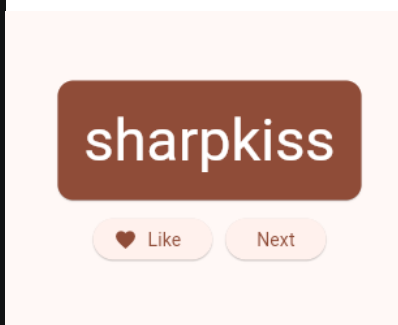
```
31  
32 class MyAppState extends ChangeNotifier {  
33   var current = WordPair.random();  
34  
35   var favorites = <WordPair>[];  
36  
37   void getNext() {  
38     current = WordPair.random();  
39     notifyListeners();  
40   }  
41  
42   void toggleFavorite() {  
43     if (favorites.contains(current)) {  
44       favorites.remove(current);  
45     } else {  
46       favorites.add(current);  
47     }  
48  
49     notifyListeners();  
50   }  
51 }
```

Validando si la palabra ya fue guardada, usando una condición para cambiar el ícono del botón dependiendo de si el par actual está en favoritos.

```
53
54 class MyHomePage extends StatelessWidget {
55   @override
56   Widget build(BuildContext context) {
57     var appState = context.watch<MyAppState>();
58     var pair = appState.current;
59
60     IconData icon;
61
62     if (appState.favorites.contains(pair)) {
63       icon = Icons.favorite;
64     } else {
65       icon = Icons.favorite_border;
66     }
67
68     return Scaffold(
```

Organizando los botones de la aplicación, colocando el botón de favoritos y el botón Next dentro de una fila para que aparezcan uno junto al otro.

```
56   Widget build(BuildContext context) {
75     SizedBox(height: 10),
76     Row(
77       mainAxisAlignment: MainAxisAlignment.min,
78       children: [
79         ElevatedButton.icon(
80           onPressed: () {
81             appState.toggleFavorite();
82           },
83           icon: Icon(icon),
84           label: Text('Like'),
85           // ElevatedButton.icon
86         ),
87         ElevatedButton(
88           onPressed: () {
89             appState.getNext();
90           },
91           child: Text('Next'),
```

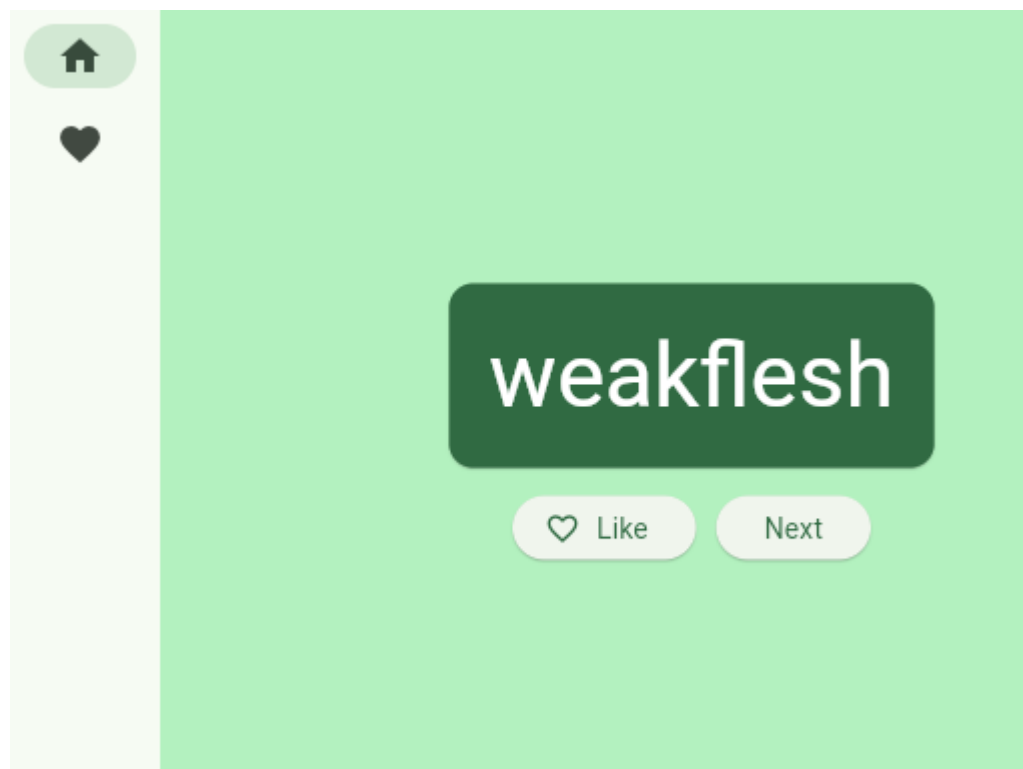


Agregando un riel de navegación

Agregando un riel de navegación, colocando un menú lateral con las opciones Home y Favorites, además de un Widget nuevo

```
SafeArea(  
  child: NavigationRail(  
    extended: false,  
    destinations: [  
      NavigationRailDestination(  
        icon: Icon(Icons.home),  
        label: Text('Home'),  
      ), // NavigationRailDestination  
      NavigationRailDestination(  
        icon: Icon(Icons.favorite),  
        label: Text('Favorites'),  
      ), // NavigationRailDestination  
    ],  
  ),  
)
```

```
90  
91 class GeneratorPage extends StatelessWidget {  
92   @override  
93   Widget build(BuildContext context) {  
94     var appState = context.watch<MyAppState>();  
95     var pair = appState.current;  
96   }
```



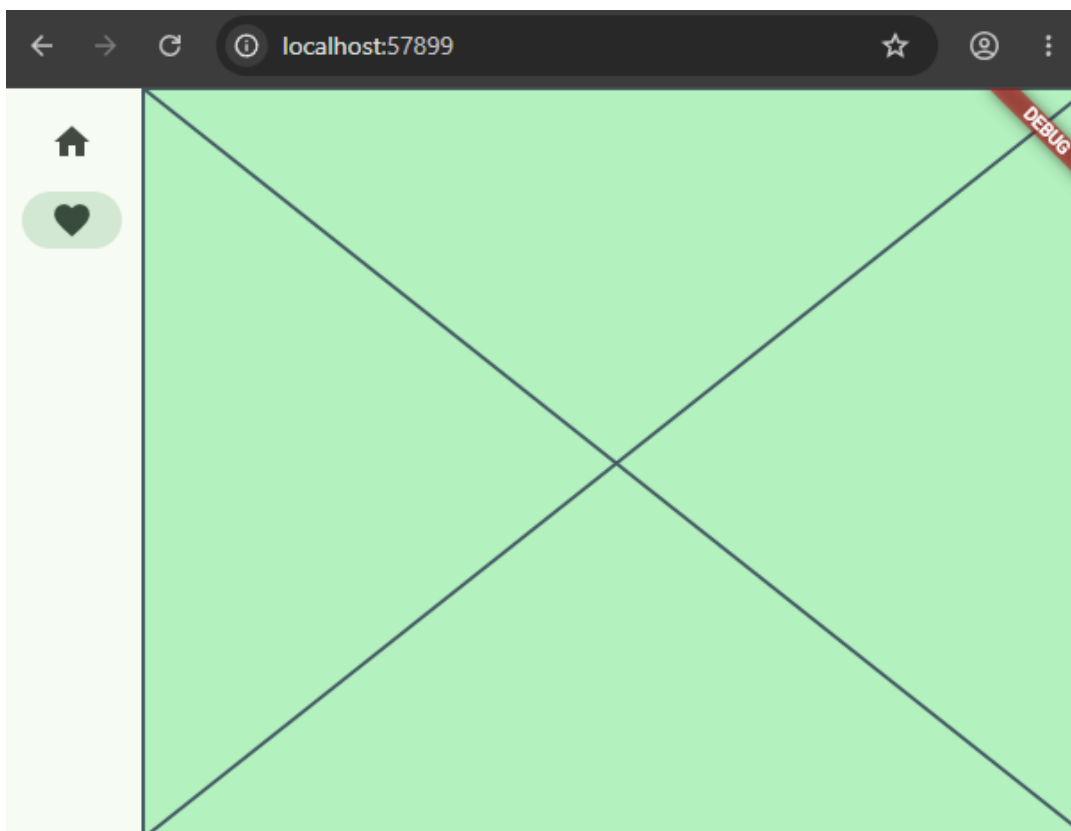
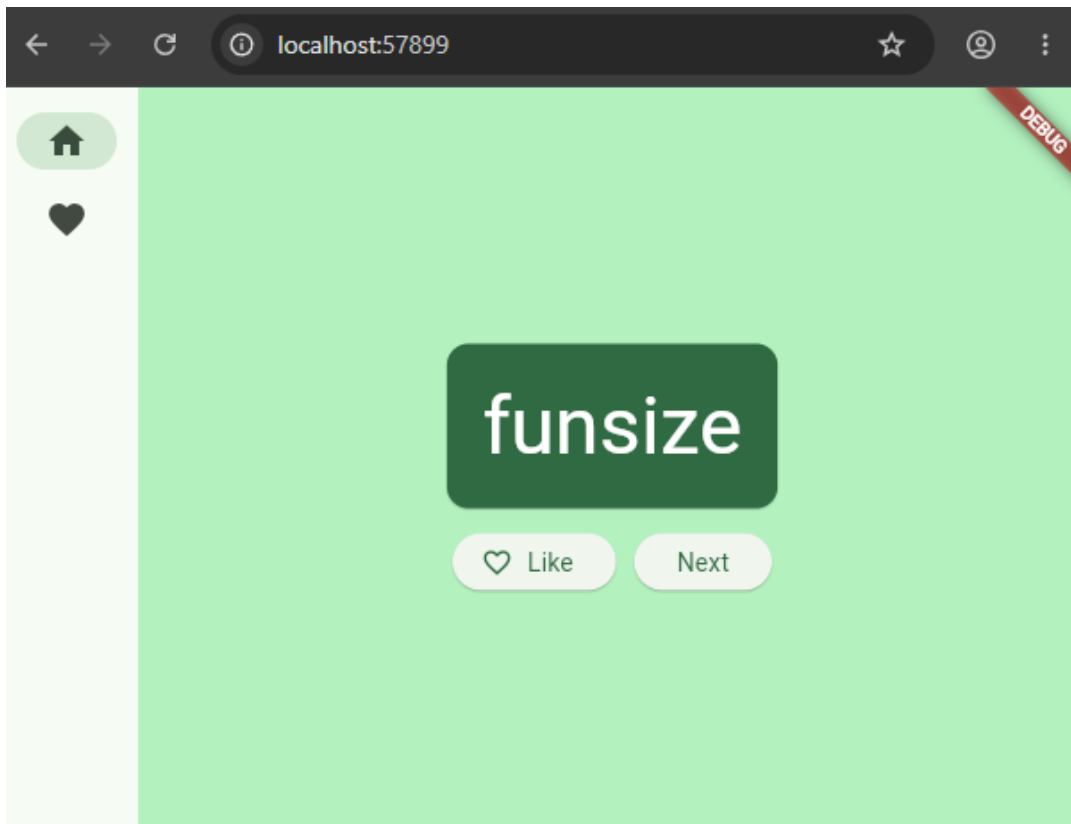
En este paso convertí MyHomePage en un StatefulWidget para poder controlar qué opción del riel de navegación está seleccionada. Esto permitió cambiar el contenido mostrado dependiendo de la sección elegida.

```
class MyHomePage extends StatefulWidget {  
  @override  
  State<MyHomePage> createState() => MyHomePage
```

```
class _MyHomePageState extends State<MyHomePage> {  
  var selectedIndex = 0;  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      body: Row(  
        children: [  
          SafeArea(  
            child: NavigationRail(  
              extended: false,  
              destinations: [  
                NavigationRailDestination(  
                  icon: Icon(Icons.home),  
                  label: Text('Home'),  
                ), // NavigationRailDestination
```

Agregué una variable para guardar el índice seleccionado del menú. Después usé esa variable para mostrar la página principal o la página de favoritos según la opción seleccionada por el usuario.

```
63      Widget build(BuildContext context) {  
64        Widget page;  
65  
66        switch (selectedIndex) {  
67          case 0:  
68            page = GeneratorPage();  
69            break;  
70          case 1:  
71            page = Placeholder();  
72            break;  
73          default:  
74            throw UnimplementedError('No widget for $selectedIndex');  
75        }  
76
```

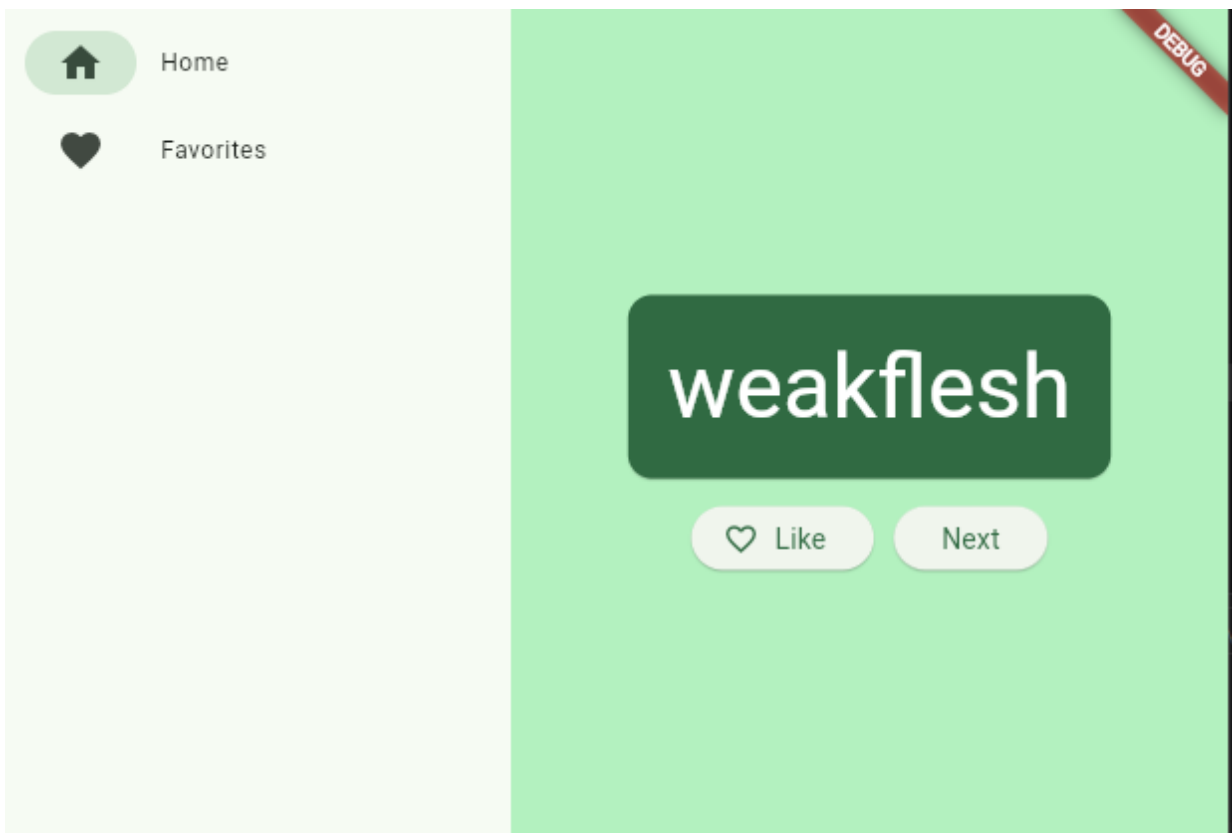


Agregando `LayoutBuilder`, haciendo que el riel de navegación se adapte automáticamente al tamaño de la pantalla.

```
76
77     return LayoutBuilder(
78       builder: (context, constraints) {
79         return Scaffold(
80           body: Row(
81             children: [
82               SafeArea(
83                 child: NavigationRail(
84                   extended: constraints.maxWidth >= 600,
85                   destinations: [
86                     NavigationRailDestination(
87                       icon: Icon(Icons.home),
```

Configurando el riel de navegación responsivo, usando el ancho de la pantalla para decidir si las etiquetas se muestran completas o solo con íconos.

```
child: NavigationRail(
  extended: constraints.maxWidth >= 600,
  destinations: [
```



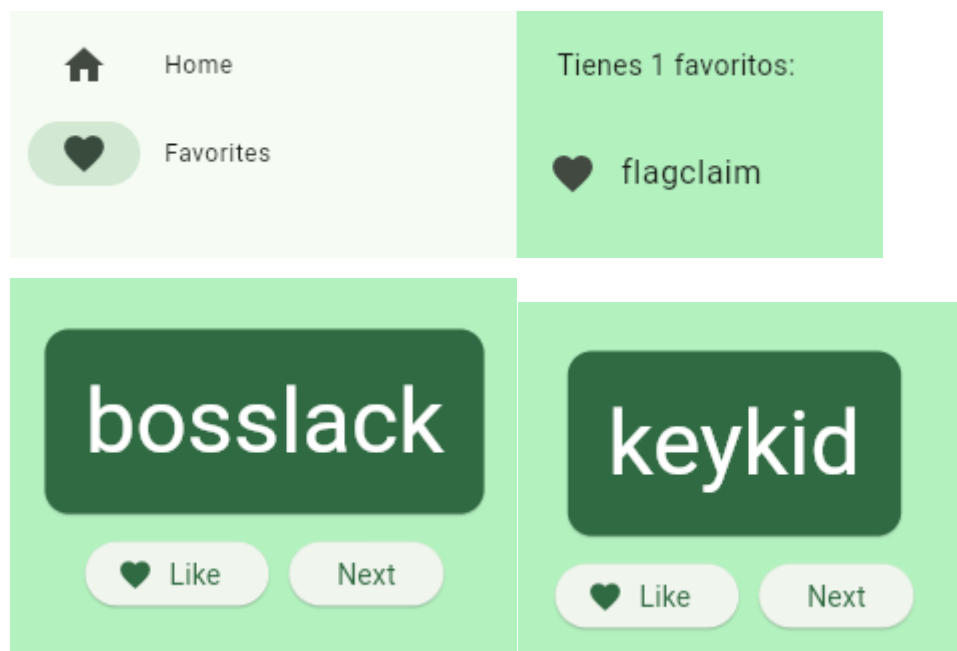
Agregando una nueva página

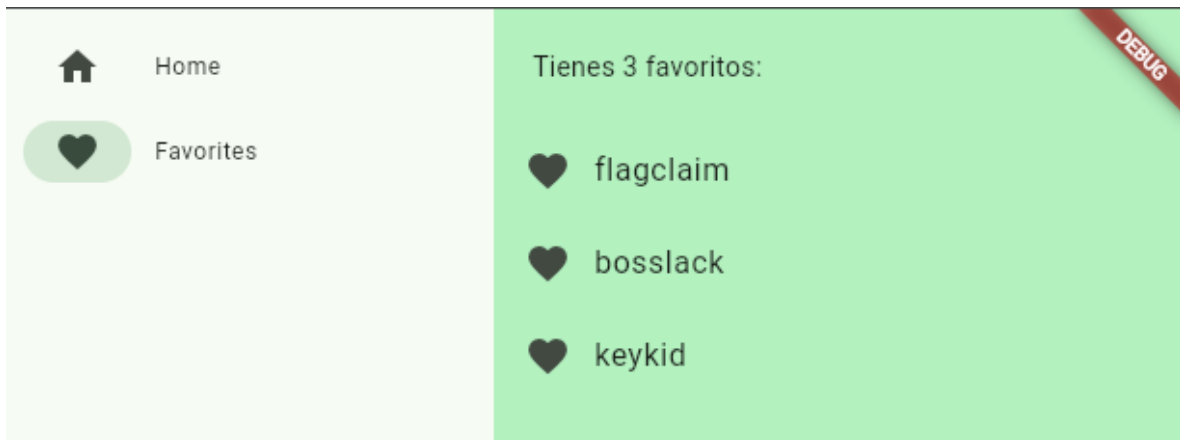
Creando una nueva página llamada FavoritesPage, donde se muestra la lista de palabras favoritas guardadas por el usuario. Si la lista está vacía, se muestra un mensaje indicando que todavía no hay favoritos.

```
161
162 class FavoritesPage extends StatelessWidget {
163   @override
164   Widget build(BuildContext context) {
165     var appState = context.watch<MyAppState>();
166
167     if (appState.favorites.isEmpty) {
168       return Center(
169         child: Text('No hay favoritos todavía.'),
170       ); // Center
171     }
172
173     return ListView(
174       children: [
```

Ya por último, solo se reemplazo el Placeholder que se puso de ejemplo por la nueva clase de la página de favoritos, añadiéndolo a la aplicación

```
break;
case 1:
  page = FavoritesPage();
break;
default:
```





CONCLUSIÓN

En esta práctica aprendí a continuar el desarrollo de una aplicación en Flutter agregando una página de favoritos funcional. También comprendí cómo usar el estado de la aplicación para guardar información, mostrar listas dinámicas y cambiar entre pantallas usando un riel de navegación. Además, pude aplicar widgets como ListView, ListTile, Padding y Center para mejorar la presentación de la información. Esta actividad me ayudó a entender mejor cómo organizar una aplicación en diferentes páginas y cómo mostrar datos guardados por el usuario dentro de la interfaz.